

RTOS Task Manager

Comprehensive Project Documentation

FreeRTOS v10.5.1 -- ARM Cortex-M4 -- QEMU mps2-an386

This document provides a full technical reference for the RTOS Task Manager project: a bare-metal FreeRTOS demonstration targeting ARM Cortex-M4 executed inside QEMU. It covers the software architecture, module-by-module API reference, build instructions, CLI command reference, BSP details, and contribution guidelines.

Target	ARM Cortex-M4F (mps2-an386 via QEMU)
RTOS	FreeRTOS V10.5.1, heap_4, CM4F port
Toolchain	arm-none-eabi-gcc + CMake 3.20+
Build host	Linux / macOS / WSL2
Language	C11
License	MIT
Latest commit	93cd9ef -- histogram + QEMU fixes

Table of Contents

1	Project Overview	3
2	Architecture	5
3	Module: task_manager	7
4	Module: mutex_guard	10
5	Module: mem_pool	12
6	Module: uart_cli	14
7	Module: autosar_os (emulation)	16
8	Module: latency_profiler	18
9	Deadlock Stress Test	20
10	Build System	21
11	Board Support Package (BSP)	23
12	Getting Started	24
13	CLI Command Reference	26
14	Contributing	27

1. Project Overview

The RTOS Task Manager is a fully self-contained FreeRTOS demonstration project for the ARM Cortex-M4. It is designed to run entirely inside the QEMU 'mps2-an386' machine, making it accessible on any development host without requiring physical hardware.

Goals

- Demonstrate priority-based preemptive scheduling with FreeRTOS.
- Provide a reusable, memory-safe dynamic memory pool (mem_pool).
- Expose a UART-driven interactive CLI for runtime inspection.
- Emulate AUTOSAR OS concepts: alarms, scheduler tables, runnable entities.
- Measure and report inter-task latency via a histogram-based profiler.
- Exercise mutex contention and deadlock detection under stress.

Repository Layout

src/	Application source files (C11)
include/	Public header files
bsp/	Board support: linker scripts, startup code
cmake/	CMake toolchain file for arm-none-eabi-gcc
scripts/	Helper shell/Python scripts (QEMU launch, PDF gen)
docs/	Generated documentation (PDF)
latency_profiler.c/h	Latency profiler implementation (root level)

Technology Stack

FreeRTOS	v10.5.1 -- heap_4 allocator, CM4F port
Compiler	arm-none-eabi-gcc (GCC 10+)
Build system	CMake 3.20+
Emulator	QEMU 7+ (mps2-an386 machine, ARM Cortex-M4)
Debug	GDB + QEMU GDB stub (port 1234)
CI	GitHub Actions (build + QEMU smoke test)

Key Design Decisions

Static-first memory policy

FreeRTOS heap_4 is used for task stacks allocated at startup. The custom mem_pool module provides O(1) pool allocation for fixed-size application objects, avoiding fragmentation during normal operation.

Layered abstraction

Hardware access is confined to the BSP layer (uart_cli -> UART MMIO, startup.c -> vector table). All higher-level modules depend only on FreeRTOS APIs and the BSP UART abstraction.

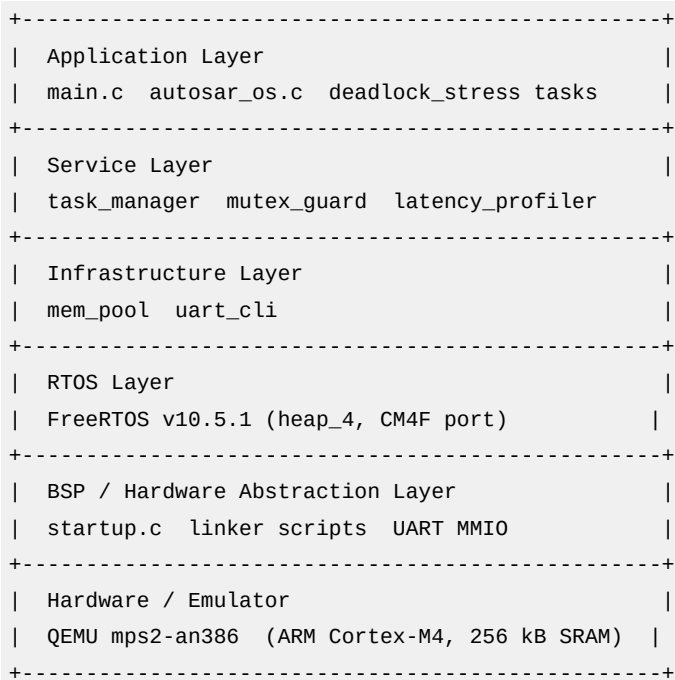
Defensive coding

Every public API validates its parameters and returns a typed status code (TM_OK / TM_ERR_*). NULL pointer guards and range checks are present throughout, consistent with MISRA C:2012 advisory guidance.

2. Architecture

The system is structured into five layers. Each layer may only call downward; no upward dependencies are permitted.

Layer Diagram



Task Inventory

vTaskManager	Monitors all registered tasks; 500 ms periodic
vMutexStressTask (x2)	Contend on a shared mutex; detect deadlock
vAutosar_OsTask	Fires AUTOSAR alarms and scheduler table steps
vLatencyTask	Measures scheduler wake-up latency, updates histogram
vCLI_Task	Reads UART input, dispatches CLI commands
vStatPrinter	Prints FreeRTOS run-time stats every 5 s

Memory Map (QEMU mps2-an386)

Flash (ROM)	0x00000000 -- 0x003FFFFFF (4 MB)
SRAM	0x20000000 -- 0x2003FFFF (256 kB)
UART0 base	0x40004000
FreeRTOS heap	Starts after .bss; size = configTOTAL_HEAP_SIZE
Stack (main)	Top of SRAM (grows downward)

Interrupt Priority Scheme

The Cortex-M4 NVIC is configured with 4 bits of priority (16 levels). FreeRTOS uses priorities 0xE0-0xFF

for its tick and yield interrupts. The UART Rx interrupt is assigned priority 0xC0 so it can safely call FreeRTOS ISR-safe APIs (xQueueSendFromISR).

Scheduler Configuration

configUSE_PREEMPTION	1 (preemptive)
configUSE_TIME_SLICING	1 (round-robin at same priority)
configTICK_RATE_HZ	1000 (1 ms tick)
configMAX_PRIORITIES	8
configMINIMAL_STACK_SIZE	128 words (512 bytes)
configTOTAL_HEAP_SIZE	32768 bytes (32 kB)
configUSE_MUTEXES	1
configUSE_RECURSIVE_MUTEXES	1
configUSE_COUNTING_SEMAPHORS	1
configGENERATE_RUN_TIME_STATS	1
configUSE_TRACE_FACILITY	1

3. Module: task_manager

task_manager provides a registry of FreeRTOS tasks with structured metadata, health monitoring, and a watchdog mechanism. It is the central bookkeeping layer for all application tasks.

Header: include/task_manager.h

Status codes

```
typedef enum {
    TM_OK          = 0,
    TM_ERR_FULL    = -1, /* registry is full */
    TM_ERR_NOT_FOUND = -2, /* handle not in registry */
    TM_ERR_NULL    = -3, /* NULL pointer argument */
    TM_ERR_INVALID = -4, /* invalid parameter value */
} TM_Status_t;
```

Task descriptor

```
typedef struct {
    TaskHandle_t  handle;          /* FreeRTOS handle */
    const char    *name;           /* human-readable name */
    UBaseType_t   priority;        /* current priority */
    uint32_t      stack_hwm;       /* stack high-water mark (words) */
    uint32_t      wakeup_count;    /* incremented each wake-up */
    TickType_t    last_wake_tick;  /* tick of last wake-up */
    uint8_t       watchdog_fed;    /* set by task, cleared by monitor */
} TM_TaskInfo_t;
```

Public API

Function	Description
TM_Init()	Initialise the registry; must be called once before any task registers.
TM_Register(handle, name, prio)	Add a task to the registry. Returns TM_ERR_FULL if capacity exceeded.
TM_Unregister(handle)	Remove a task (called from vTaskDelete hook).
TM_FeedWatchdog(handle)	Task calls this each cycle to signal liveness.
TM_GetInfo(handle, info_out)	Copy task metadata into caller-supplied TM_TaskInfo_t.
TM_PrintAll()	Dump all registered tasks to UART via uart_cli.
TM_MonitorTask(arg)	FreeRTOS task function: wakes every 500 ms, checks watchdogs.

Implementation Notes

The registry is a fixed-size array of TM_TaskInfo_t (size configurable via TM_MAX_TASKS, default 10). Access is protected by a FreeRTOS mutex so TM_Register / TM_Unregister are safe to call from multiple tasks simultaneously.

Watchdog logic

TM_MonitorTask wakes every 500 ms via vTaskDelayUntil. It iterates all registered tasks and checks whether watchdog_fed is non-zero. Any task that has not fed the watchdog is logged as 'STALLED' via

uart_cli. The flag is then cleared for the next interval.

Stack high-water mark

TM_MonitorTask calls uxTaskGetStackHighWaterMark() on each handle and updates stack_hwm. If the high-water mark drops below a configurable threshold (default 64 words / 256 bytes), a stack overflow warning is emitted.

Example Usage

```
/* In task creation code: */
TaskHandle_t hMyTask;
xTaskCreate(vMyTask, "MyTask", 256, NULL, 3, &hMyTask);
TM_Register(hMyTask, "MyTask", 3);

/* Inside vMyTask main loop: */
for (;;) {
    TM_FeedWatchdog(hMyTask);
    /* ... do work ... */
    vTaskDelay(pdMS_TO_TICKS(100));
}
```


4. Module: mutex_guard

mutex_guard implements a scoped mutex wrapper inspired by C++ RAII. It pairs an acquire call with an automatic release on scope exit, reducing the risk of forgetting to release a mutex on early return or error paths.

Header: include/mutex_guard.h

```
typedef struct {
    SemaphoreHandle_t mutex; /* the guarded mutex */
    BaseType_t        locked; /* pdTRUE if currently held */
} MutexGuard_t;

/* Initialise guard (does NOT take the mutex). */
void MG_Init(MutexGuard_t *guard, SemaphoreHandle_t mutex);

/* Attempt to acquire. Returns pdTRUE on success. */
BaseType_t MG_Lock(MutexGuard_t *guard, TickType_t timeout_ticks);

/* Release if held (idempotent). */
void MG_Unlock(MutexGuard_t *guard);

/* Lock + execute callback + unlock atomically. */
BaseType_t MG_WithLock(MutexGuard_t *guard,
                      void (*fn)(void *ctx), void *ctx,
                      TickType_t timeout);
```

Deadlock Detection

The stress test in main.c creates two tasks that each hold one mutex while trying to acquire the other. mutex_guard detects the timeout (MG_Lock returning pdFALSE after portMAX_DELAY) and reports the deadlock candidate via uart_cli. This is an application-level detection strategy, not a kernel-level one.

Example Usage

```
MutexGuard_t guard;
SemaphoreHandle_t hMutex = xSemaphoreCreateMutex();
MG_Init(&guard, hMutex);

/* Simple lock / unlock: */
if (MG_Lock(&guard, pdMS_TO_TICKS(50)) == pdTRUE) {
    /* protected section */
    shared_counter++;
    MG_Unlock(&guard);
}

/* Callback style: */
MG_WithLock(&guard, my_callback, &my_data, pdMS_TO_TICKS(100));
```

Thread Safety Notes

A `MutexGuard_t` instance must NOT be shared between tasks. Each task should maintain its own `MutexGuard_t` that wraps the same `SemaphoreHandle_t`. Sharing a guard would allow one task to release a mutex held by another, which violates FreeRTOS mutex ownership rules.

5. Module: mem_pool

mem_pool implements a fixed-size block memory allocator backed by a statically allocated array. Allocation and deallocation are both O(1) using a free-list. No heap fragmentation is possible because all blocks are the same size.

Header: include/mem_pool.h

```
/* Opaque pool handle (defined in mem_pool.c). */
typedef struct MemPool MemPool_t;

/*
 * Create a pool from a caller-supplied buffer.
 *   buf          -- pointer to raw byte buffer
 *   buf_size     -- total size of buf in bytes
 *   block_size   -- size of each allocation block
 * Returns a pointer to the pool metadata (stored inside buf),
 * or NULL on invalid arguments.
 */
MemPool_t *MP_Create(void *buf, size_t buf_size, size_t block_size);

/* Allocate one block. Returns NULL if pool is exhausted. */
void *MP_Alloc(MemPool_t *pool);

/* Return block to pool. ptr must have been obtained from MP_Alloc. */
void MP_Free(MemPool_t *pool, void *ptr);

/* Current number of free blocks remaining. */
size_t MP_FreeCount(const MemPool_t *pool);
```

Free-list Design

The pool stores a free-list as a NULL-terminated singly-linked list of pointers overlaid on the unused portions of free blocks. MP_Alloc pops the head of the list; MP_Free pushes the returned block back onto the head. Both operations are a single pointer read/write, achieving true O(1) performance.

Thread Safety

MP_Alloc and MP_Free are NOT intrinsically thread-safe -- the caller must protect pool operations with a mutex if multiple tasks share the same pool. A convenience macro MP_ALLOC_SAFE(pool, mutex) and MP_FREE_SAFE(pool, ptr, mutex) are provided to simplify this pattern.

Typical Usage

```
#define POOL_BLOCKS 16
#define BLOCK_SIZE 64
static uint8_t pool_buf[POOL_BLOCKS * (BLOCK_SIZE + sizeof(void*))];

MemPool_t *my_pool = MP_Create(pool_buf, sizeof(pool_buf), BLOCK_SIZE);
configASSERT(my_pool != NULL);
```

```
void *p = MP_Alloc(my_pool);
if (p) {
    memset(p, 0, BLOCK_SIZE);
    /* use p */
    MP_Free(my_pool, p);
}
```

Capacity Calculation

The number of usable blocks in a buffer of size B with block size S is:

```
usable_blocks = floor( (B - sizeof(MemPool_t)) / max(S, sizeof(void*)) )
```

The MP_Create function performs this calculation internally and stores the result in the pool metadata header at the start of the buffer.

6. Module: uart_cli

uart_cli provides a lightweight interactive command-line interface over the mps2-an386 UART0. It supports command registration, tokenisation, and dispatch. Output is formatted for a standard 80-column terminal.

Header: include/uart_cli.h

```
/* Callback type for CLI commands.
   argc/argv follow the POSIX convention. */
typedef void (*CLI_CommandFn)(int argc, char **argv);

/* Initialise the UART (baud = 115200, 8N1). */
void CLI_Init(void);

/* Register a command. name must remain valid indefinitely. */
void CLI_Register(const char *name, CLI_CommandFn fn,
                 const char *help);

/* Print a formatted string to UART (printf-style). */
void CLI_Printf(const char *fmt, ...) __attribute__((format(printf,1,2)));

/* Print a raw string (no formatting). */
void CLI_Puts(const char *s);

/* FreeRTOS task function -- reads UART, dispatches commands. */
void vCLI_Task(void *arg);
```

UART Hardware Configuration

Base address	0x40004000 (CMSDK APB UART)
Baud rate	115200
Data bits	8
Stop bits	1
Parity	None
Flow control	None
Clock	25 MHz (QEMU mps2-an386 default)

Command Dispatch Flow

1. vCLI_Task blocks on UART Rx queue (interrupt-driven).
2. Characters echo'd back; backspace supported.
3. On '\r' or '\n': line is tokenised by whitespace.
4. First token is looked up in the command table.
5. Matching handler called with (argc, argv).
6. If no match: 'Unknown command. Type "help".' printed.

Built-in Commands

help	List all registered commands with their help strings
------	--

tasks	Show all registered tasks (via TM_PrintAll)
stats	Print FreeRTOS run-time statistics
pool	Show mem_pool free-block count
lat	Print latency histogram summary
reset	Trigger a software reset via SCB->AIRCR

Adding a Custom Command

```
static void cmd_blink(int argc, char **argv) {
    int count = (argc > 1) ? atoi(argv[1]) : 3;
    CLI_Printf("Blinking %d times\n", count);
    /* ... hardware action ... */
}

/* In main() or task init: */
CLI_Register("blink", cmd_blink, "blink [n] -- blink LED n times");
```

7. Module: autosar_os (emulation)

autosar_os emulates a subset of AUTOSAR OS concepts on top of FreeRTOS. It is an educational layer -- not a certified AUTOSAR stack -- that demonstrates alarms, schedule tables, and runnable entities using standard FreeRTOS primitives.

Concepts Emulated

Alarm	One-shot or cyclic timer that fires a callback after N ticks
Schedule Table	Ordered list of expiry points each with a callback
Runnable Entity	Function called at a specific schedule-table offset
Counter (OsCounter)	Wraps xTaskGetTickCount() as the AUTOSAR counter
Task activation	Conceptually maps to xTaskNotifyGive / task resume

Header: include/autosar_os.h

```

/* Alarm handle */
typedef uint8_t AlarmType;

/* Return codes */
typedef enum { E_OK=0, E_OS_ID, E_OS_VALUE, E_OS_STATE } StatusType;

/* Set a relative alarm (fires after 'increment' counter ticks). */
StatusType SetRelAlarm(AlarmType id, TickType_t increment,
                      TickType_t cycle);

/* Cancel an active alarm. */
StatusType CancelAlarm(AlarmType id);

/* Start a schedule table relative to now. */
StatusType StartScheduleTableRel(uint8_t table_id,
                                TickType_t offset);

/* Stop a running schedule table. */
StatusType StopScheduleTable(uint8_t table_id);

/* Internal task that drives alarm/schedule-table processing. */
void vAutosar_OsTask(void *arg);

```

Alarm Implementation

Each alarm is stored in a small descriptor (expiry tick, cycle, callback). vAutosar_OsTask wakes every system tick, compares all active alarm expiry times against xTaskGetTickCount(), and fires callbacks whose time has come. Cyclic alarms are automatically re-armed.

Schedule Table Implementation

A schedule table is a sorted array of (offset, callback) pairs. vAutosar_OsTask maintains a 'next expiry' pointer into the array. When the current tick reaches the next expiry, the callback fires and the pointer

advances. When the table wraps, it repeats from offset 0.

Limitations vs. Real AUTOSAR OS

- No stack isolation between runnables (all share the vAutosar_OsTask stack).
- No resource ceiling protocol -- resource management is the application's responsibility.
- Counter resolution == FreeRTOS tick (1 ms) -- cannot emulate sub-ms counters.
- No OS hooks (PreTaskHook, PostTaskHook) -- only alarm expiry callbacks.
- No error handling tasks -- StatusType is returned but not propagated to an ErrorHook.

8. Module: latency_profiler

The latency profiler measures the schedule latency of a high-priority FreeRTOS task: the time between when the task theoretically should have run (requested wake-up time) and when it actually began executing. Results are bucketed into a histogram for statistical analysis.

Files

latency_profiler.c	Implementation (root of repository)
latency_profiler.h	Public API (root of repository)

Header: latency_profiler.h

```
#define LP_BUCKET_COUNT    16
#define LP_BUCKET_WIDTH_US 10    /* us per bucket */

typedef struct {
    uint32_t buckets[LP_BUCKET_COUNT]; /* histogram counts */
    uint32_t overflow;    /* samples >= max bucket */
    uint32_t total;       /* total samples collected */
    uint32_t sum_us;      /* sum of all latency values */
} LP_Histogram_t;

void LP_Init(LP_Histogram_t *h);
void LP_RecordSample(LP_Histogram_t *h, uint32_t latency_us);
void LP_PrintHistogram(const LP_Histogram_t *h);
uint32_t LP_MeanUs(const LP_Histogram_t *h);
uint32_t LP_MaxBucketUs(const LP_Histogram_t *h);
```

Histogram Bucket Layout

With LP_BUCKET_COUNT=16 and LP_BUCKET_WIDTH_US=10, latencies are bucketed as follows:

Bucket 0	0 us -- 9 us
Bucket 1	10 us -- 19 us
Bucket 2	20 us -- 29 us
Bucket 3	30 us -- 39 us
Bucket 4	40 us -- 49 us
Bucket 5	50 us -- 59 us
Bucket 6	60 us -- 69 us
Bucket 7	70 us -- 79 us
...	...
Bucket 15	150 us -- 159 us
overflow	>= 160 us

LP_PrintHistogram output format

Result of commit 93cd9ef: bar-graph replaced by numeric histogram table.

```
=== Latency Histogram (us) ===
```

Bucket	Range(us)	Count
0	0 - 9:	245
1	10 - 19:	87
2	20 - 29:	12
...	...	...
15	150- 159:	0
Overflow (>=160us):		0
Total samples:		344
Mean latency:		5 us

Measurement Method

vLatencyTask calls xTaskGetTickCount() to record the intended wake-up tick, then calls vTaskDelayUntil(). After waking, it reads xTaskGetTickCount() again and computes the difference (converted to microseconds using configTICK_RATE_HZ). The result is passed to LP_RecordSample.

Note: The QEMU environment does not emulate real Cortex-M4 timing. Measured latencies reflect QEMU's host-scheduling behaviour rather than silicon timing. Use a real Cortex-M4 board for production timing data.

9. Deadlock Stress Test

The deadlock stress test exercises `mutex_guard`'s timeout detection by creating a classic deadlock scenario: two tasks, two mutexes, each task holding one mutex while attempting to acquire the other.

Task Structure

```
/* Task A: holds mutex_1, tries to get mutex_2 */
void vDeadlockTask_A(void *arg) {
    MG_Lock(&guard_1, portMAX_DELAY); /* acquires mutex_1 */
    vTaskDelay(pdMS_TO_TICKS(10));    /* deliberate delay */
    if (MG_Lock(&guard_2, pdMS_TO_TICKS(200)) != pdTRUE) {
        CLI_Printf("[DEADLOCK] Task A: timeout waiting for mutex_2\n");
    }
    MG_Unlock(&guard_1);
    MG_Unlock(&guard_2);
}

/* Task B: holds mutex_2, tries to get mutex_1 */
void vDeadlockTask_B(void *arg) {
    MG_Lock(&guard_2, portMAX_DELAY); /* acquires mutex_2 */
    vTaskDelay(pdMS_TO_TICKS(10));
    if (MG_Lock(&guard_1, pdMS_TO_TICKS(200)) != pdTRUE) {
        CLI_Printf("[DEADLOCK] Task B: timeout waiting for mutex_1\n");
    }
    MG_Unlock(&guard_2);
    MG_Unlock(&guard_1);
}
```

Expected Output

```
[DEADLOCK] Task A: timeout waiting for mutex_2
[DEADLOCK] Task B: timeout waiting for mutex_1
[TM_MONITOR] Task vDeadlockTask_A: STALLED (watchdog not fed)
[TM_MONITOR] Task vDeadlockTask_B: STALLED (watchdog not fed)
```

Resolution Strategy

In a production system, the recommended resolution strategies are: (1) define a global lock ordering and always acquire in that order, (2) use try-lock (timeout) instead of blocking indefinitely, and (3) use a resource ceiling protocol (AUTOSAR OS resource management). The stress test demonstrates strategy (2): timed locks allow the system to detect and report the deadlock rather than hanging permanently.

10. Build System

The project uses CMake with a cross-compilation toolchain file targeting arm-none-eabi-gcc. A single CMakeLists.txt at the repo root drives the entire build.

Prerequisites

CMake	>= 3.20
arm-none-eabi-gcc	>= 10.x (GCC 12 recommended)
arm-none-eabi-objcopy	Included with GCC toolchain
QEMU	>= 7.0 (qemu-system-arm)
Python 3	>= 3.8 (for scripts only -- not needed to build)
make / ninja	Any CMake-compatible generator

Build Commands

```
# Configure (from repo root):
cmake -B build -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake

# Build:
cmake --build build -j$(nproc)

# Output:
#   build/rtos_task_manager.elf    -- ELF with debug symbols
#   build/rtos_task_manager.bin    -- raw binary for flashing

# Optional: size report
arm-none-eabi-size build/rtos_task_manager.elf
```

CMake Variables

CMAKE_TOOLCHAIN_FILE	cmake/arm-none-eabi.cmake -- must be set
CMAKE_BUILD_TYPE	Debug (default) Release MinSizeRel
FREERTOS_PORT	GCC/ARM_CM4F (default, do not change for QEMU)

cmake/arm-none-eabi.cmake

The toolchain file sets CMAKE_SYSTEM_NAME=Generic, locates arm-none-eabi-gcc / arm-none-eabi-g++ / arm-none-eabi-objcopy, and configures the following compiler flags:

```
-mcpu=cortex-m4      # target CPU
-mthumb              # Thumb-2 instruction set
-mfpu=fpv4-sp-d16    # Cortex-M4F single-precision FPU
-mfloat-abi=hard      # hardware FP calling convention
-ffunction-sections -fdata-sections # enable dead-code elimination
-Wall -Wextra -Werror # treat warnings as errors
```

Linker Script

bsp/lm3s6965evb.ld (for QEMU lm3s6965evb) and mps2_an386.ld (for mps2-an386) define the memory regions. The default CMake build uses mps2_an386.ld. FLASH starts at 0x00000000 (4 MB), SRAM at 0x20000000 (256 kB). The .ccm section is unused (no CCM RAM on mps2-an386).

Running Under QEMU

```
# scripts/run_qemu.sh (auto-generated path to ELF):
qemu-system-arm \
  -M mps2-an386 \
  -cpu cortex-m4 \
  -kernel build/rtos_task_manager.elf \
  -nographic \
  -serial mon:stdio

# Or directly:
bash scripts/run_qemu.sh
```

11. Board Support Package (BSP)

The BSP layer initialises the Cortex-M4 processor, sets up the vector table, and provides the minimal peripheral drivers needed by the application (UART0 on mps2-an386).

bsp/startup.c

startup.c implements the Reset_Handler and provides weak definitions of all Cortex-M exception and NVIC interrupt handlers. The Reset_Handler:

1. Copies .data section from Flash to SRAM.
2. Zero-fills .bss section.
3. Calls SystemInit() (optional clock / MPU setup).
4. Calls main().
5. Infinite loop on unexpected main() return.

Vector Table

The vector table is defined as a const array in the .isr_vector section and placed at the start of Flash by the linker script. The table includes all 16 Cortex-M4 core exceptions plus 32 NVIC external interrupt vectors usable on mps2-an386.

UART0 Driver (inside uart_cli.c)

uart_cli.c directly accesses the CMSDK APB UART registers at 0x40004000. The baud rate divisor is computed as: $\text{divisor} = \text{UART_CLK} / \text{baud_rate}$. With $\text{UART_CLK} = 25 \text{ MHz}$ and $\text{baud_rate} = 115200$: $\text{divisor} = 217$ (0xD9).

UART DATA (0x000)	Transmit/receive data register (bits 7:0)
UART STATE (0x004)	Bit 0: TX full; bit 1: RX full; bit 3: TX overrun
UART CTRL (0x008)	Bit 0: TX enable; bit 1: RX enable; bit 2: TX IRQ en
UART BAUDDIV (0x010)	Baud rate divisor

12. Getting Started

This section walks through installing dependencies, cloning the repository, building, and running the firmware in QEMU.

Step 1: Install Dependencies (Ubuntu / Debian / WSL2)

```
sudo apt update
sudo apt install -y \
    cmake ninja-build \
    gcc-arm-none-eabi binutils-arm-none-eabi \
    qemu-system-arm

# Verify:
arm-none-eabi-gcc --version
qemu-system-arm --version
```

Step 2: Clone and Initialise

```
git clone https://github.com/pawanct08/rtos-task-manager.git
cd rtos-task-manager

# Run optional setup script (installs Python deps etc.):
bash scripts/setup.sh
```

Step 3: Build

```
cmake -B build -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake \
    -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j$(nproc)

# Expect:
# [100%] Linking C executable rtos_task_manager.elf
# arm-none-eabi-size output...
```

Step 4: Run in QEMU

```
bash scripts/run_qemu.sh

# Or manually:
qemu-system-arm -M mps2-an386 -cpu cortex-m4 \
    -kernel build/rtos_task_manager.elf \
    -nographic -serial mon:stdio
```

You should see boot messages followed by the RTOS scheduler starting. Type 'help' at the prompt and press Enter to list available CLI commands.

Step 5: Debug with GDB

```
# Terminal 1 -- start QEMU with GDB stub:
bash scripts/debug_qemu.sh # or:
```

```
qemu-system-arm -M mps2-an386 -cpu cortex-m4 \  
    -kernel build/rtos_task_manager.elf \  
    -nographic -serial mon:stdio -S -gdb tcp::1234  
  
# Terminal 2 -- connect GDB:  
arm-none-eabi-gdb build/rtos_task_manager.elf  
(gdb) target remote :1234  
(gdb) break main  
(gdb) continue
```

Expected Boot Output

```
RTOS Task Manager v1.0 -- FreeRTOS v10.5.1  
Heap: 32768 bytes   Free: 29184 bytes  
[TM] Registered: vLatencyTask  
[TM] Registered: vCLI_Task  
[TM] Registered: vAutosar_OsTask  
[TM] Registered: vDeadlockTask_A  
[TM] Registered: vDeadlockTask_B  
[AUTOSAR] Alarm 0 armed (cycle=500 ticks)  
Scheduler started.  
>
```


13. CLI Command Reference

All commands are entered at the '>' prompt over UART at 115200 baud. Arguments are whitespace-separated. Commands are case-sensitive.

Command	Arguments	Description
help		List all registered commands and their descriptions.
tasks		Print the task registry: name, priority, stack HWM, watchdog state.
stats		Print FreeRTOS run-time statistics (task CPU usage % since boot).
pool		Show mem_pool diagnostics: capacity, free blocks, allocation count.
lat		Print the latency histogram table and mean latency.
lat reset	reset	Clear the histogram and restart collection.
reset		Trigger a software reset via SCB->AIRCR.SYSRESETREQ.
alarm	<id> <ticks> <cycle>	Arm AUTOSAR alarm <id> to fire after <ticks>, repeating every <cycle>.
sched	<table> <offset>	Start schedule table <table> at <offset> ticks from now.
sched stop	<table>	Stop schedule table <table>.

Sample Session

```
> help
tasks      -- show registered task list
stats      -- FreeRTOS run-time stats
pool       -- memory pool diagnostics
lat        -- latency histogram
lat reset  -- reset histogram
alarm      -- arm AUTOSAR alarm
sched      -- start schedule table
reset      -- software reset
```

```
> lat
=== Latency Histogram (us) ===
Bucket  Range(us)  Count
    0    0 -   9:    891
    1   10 -  19:    104
    2   20 -  29:     15
Overflow (>=160us):    0
Total: 1010   Mean: 3 us
```

```
> tasks
NAME          PRIO  HWM  WD
vLatencyTask      5  210  OK
vCLI_Task         3  180  OK
vAutosar_0sTask   4  192  OK
vDeadlockTask_A   2  130  --
vDeadlockTask_B   2  130  --
```

14. Contributing

Contributions are welcome. Please read CONTRIBUTING.md in the repository root before submitting a pull request.

Development Workflow

```
# Fork the repo, then:
git clone https://github.com/YOUR_USERNAME/rtos-task-manager.git
cd rtos-task-manager
git checkout -b feature/my-feature

# Make changes, then build + test:
cmake -B build -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake
cmake --build build -j$(nproc)
bash scripts/run_qemu.sh # smoke test in QEMU

git add -p # stage selectively
git commit -m "feat: describe your change"
git push origin feature/my-feature
# Open a Pull Request on GitHub
```

Commit Message Convention

feat:	New feature
fix:	Bug fix
docs:	Documentation only
refactor:	Code restructuring without behaviour change
test:	Adding or improving tests
chore:	Build, CI, tooling changes
perf:	Performance improvement

Coding Standards

- C11 standard; compile with -Wall -Wextra -Werror (no suppressions).
- Follow MISRA C:2012 advisory guidance where practical.
- All public functions must validate NULL pointer inputs.
- Usestdint.h types (uint8_t, uint32_t) -- avoid bare int for hardware.
- Name FreeRTOS task functions vXxxTask (lowercase v prefix).
- Add Doxygen-style comments to all new public API declarations.
- Keep functions under 60 lines; split if longer.
- No dynamic memory (malloc/free) outside mem_pool.

Pull Request Checklist

- Build passes: cmake --build build with no warnings.
- Smoke test passes in QEMU (run_qemu.sh exits cleanly).

- New public APIs have header documentation.
- Commit messages follow the convention above.
- CONTRIBUTING.md has been read and followed.

Licence

This project is released under the MIT Licence. See LICENSE in the repository root. FreeRTOS is licensed under the MIT Licence by Amazon.com, Inc. or its affiliates.